

HTML Interview Preparation Guide

A complete reference — concepts, code, and the "why" interviewers actually probe for.

Table of Contents

1. [HTML Basics](#)
 2. [Important Tags](#)
 3. [Semantic HTML](#)
 4. [Forms](#)
 5. [Interview Questions & Answers](#)
 6. [Quick-Fire Revision Table](#)
-

1. HTML Basics

1.1 What is HTML?

HTML (HyperText Markup Language) is the standard markup language used to structure content on the web. It is **not a programming language** — it has no logic, loops, or conditionals. It simply describes *what* content is (a heading, a paragraph, a list, an image) using **tags**.

- **HyperText** → text containing links to other text/pages (hyperlinks).
- **Markup Language** → uses tags to annotate/structure content, rather than executing instructions.

Together with **CSS** (styling/presentation) and **JavaScript** (behavior/interactivity), HTML forms the three core building blocks of the web.

Interview framing: If asked "Is HTML a programming language?" — say no, and explain *why* (no variables, no control flow, it's declarative markup) rather than just stating a fact.

1.2 HTML Document Structure

```
html
```

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Page Title</title>
</head>
<body>
  <h1>Hello World</h1>
  <p>This is the visible content.</p>
</body>
</html>

```

Part	Purpose
<code><!DOCTYPE html></code>	Tells the browser to render in standards mode (HTML5). Without it, browsers fall into "quirks mode" and rendering can break in inconsistent, legacy ways.
<code><html lang="en"></code>	Root element wrapping the entire document. <code>lang</code> attribute helps screen readers and search engines.
<code><head></code>	Container for metadata — not rendered on the page itself.
<code><body></code>	Container for everything that's visible/rendered to the user.

1.3 Tags, Elements, Attributes

These three terms get mixed up constantly in interviews — know the precise distinction:

- **Tag** — the markup keyword wrapped in angle brackets. E.g. `<p>` (opening tag), `</p>` (closing tag).
- **Element** — the tag *plus* its content (and closing tag, if it has one). E.g. `<p>Hello</p>` is an element. Some elements are **self-closing** / **void elements** (no content, no closing tag) — e.g. `<img>`, `<br>`, `<input>`, `<hr>`.
- **Attribute** — extra information provided inside the opening tag, as `name="value"` pairs. E.g. in ``, `src` and `alt` are attributes.

html

```
<a href="https://example.com" target="_blank">Visit</a>
```

- Tag: `<a>`

- Element: the whole `<a>...</a>` block
- Attributes: `href`, `target`

1.4 Head vs Body

`<head>`

`<body>`

Not visible on the page

Everything visible on the page

Contains `<title>`, `<meta>`, `<link>` (CSS), sometimes `<script>`

Contains headings, paragraphs, images, forms, etc.

Read by browsers, search engines, social media crawlers

Rendered and interacted with by the user

Common `<head>` contents:

html

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Site</title>
  <link rel="stylesheet" href="styles.css">
  <meta name="description" content="Short page description">
</head>
```

2. Important Tags

`<h1>` – `<h6>`

Headings, in decreasing order of importance/size. `<h1>` is the most important (main title), `<h6>` the least.

html

```
<h1>Main Title</h1>
<h2>Section Heading</h2>
```

Interview note: Best practice is one `<h1>` per page (the main topic), then nest `<h2>` – `<h6>` logically like an outline. This matters for **SEO** and **accessibility** (screen reader users can jump between headings).

<p>

Defines a paragraph of text. Browsers automatically add margin above/below it.

html

```
<p>This is a paragraph of text.</p>
```

<div>

A generic **block-level** container with no semantic meaning — purely for grouping content, usually to apply CSS layout/styling or to target with JavaScript.

html

```
<div class="card">  
  <h2>Title</h2>  
  <p>Some text</p>  
</div>
```

A generic **inline** container with no semantic meaning — used to style or target a small piece of text *within* a line, without breaking the flow.

html

```
<p>This is <span class="highlight">important</span> text.</p>
```

Embeds an image. Self-closing (void element).

html

```
**

Anchor tag — creates a hyperlink.

html

```
Visit Site
```

Key attributes: `href` (destination), `target="_blank"` (opens in new tab), `rel="noopener noreferrer"` (security best practice when using `target="_blank"`, prevents the new page from accessing `window.opener`).

`<ul>`, `<ol>`, `<li>`

Lists. `<ul>` = unordered (bullets), `<ol>` = ordered (numbers), `<li>` = each list item, used inside either.

html

```

 Apple
 Banana

 First step
 Second step

```

`<table>`

Displays tabular data. Built from `<thead>` (header row), `<tbody>` (body rows), `<tr>` (table row), `<th>` (header cell), `<td>` (data cell).

html

```
<table>
 <thead>
 <tr><th>Name</th><th>Age</th></tr>
 </thead>
 <tbody>
 <tr><td>Chetan</td><td>21</td></tr>
 </tbody>
</table>
```

**Interview note:** Tables should only be used for genuinely tabular data — not for page layout (that's what CSS Grid/Flexbox are for; using tables for layout is an old, deprecated practice).

## <form>

Container for interactive input controls, used to collect and submit user data.

html

```
<form action="/submit" method="POST">
 <input type="text" name="username">
 <button type="submit">Submit</button>
</form>
```

Key attributes: `action` (URL to send data to), `method` (`GET` or `POST` — see Section 4).

## <input>

The most versatile form control — its behavior changes entirely based on the `type` attribute (text, email, checkbox, radio, date, file, etc. — full list in Section 4).

html

```
<input type="email" name="email" placeholder="you@example.com" required>
```

## <button>

A clickable button. Has three possible `type` values:

- `type="submit"` (default inside a form) — submits the form
- `type="reset"` — clears the form
- `type="button"` — does nothing by default; used for custom JS behavior

html

```
<button type="submit">Send</button>
```

## <textarea>

A multi-line text input (unlike `<input type="text">`, which is single-line).

html

```
<textarea rows="4" cols="50" placeholder="Write your message..."></textarea>
```

## <select>

A dropdown menu, built from `<option>` elements.

html

```
<select name="country">
 <option value="in">India</option>
 <option value="us">USA</option>
</select>
```

**<iframe>**

Embeds another HTML document/page inside the current page — commonly used for YouTube videos, Google Maps, payment widgets, etc.

html

```
<iframe src="https://www.youtube.com/embed/xyz" width="560" height="315"></iframe>
```

**Interview note:** **<iframe>** has security implications — cross-origin content in an iframe is sandboxed by the browser's Same-Origin Policy. The **<sandbox>** attribute can further restrict what an embedded iframe is allowed to do (no scripts, no forms, no popups, etc.).

---

### 3. Semantic HTML

What "semantic" means here

A **semantic tag** clearly describes its *meaning* and *role* in the content — as opposed to **<div>**/**<span>**, which say nothing about what the content actually is.

**<header>**

Introductory content for a page or a section — typically a logo, site title, or navigation.

html

```
<header>
 <h1>My Blog</h1>
 <nav>...</nav>
</header>
```

**<nav>**

Wraps a block of navigation links (main menu, breadcrumbs, table of contents).

html

```
<nav>
 Home
 About
</nav>
```

### <section>

A thematic grouping of content, generally with its own heading. Think of it as a distinct "chapter" of a page.

html

```
<section>
 <h2>About Us</h2>
 <p>...</p>
</section>
```

### <article>

Self-contained content that would make sense on its own, independent of the rest of the page — e.g. a blog post, a news story, a forum post.

html

```
<article>
 <h2>Blog Post Title</h2>
 <p>Content...</p>
</article>
```

### <aside>

Content tangentially related to the main content — sidebars, pull quotes, related links, ads.

html

```
<aside>
 <h3>Related Posts</h3>
 ...
</aside>
```

### <footer>

Closing content for a page or section — copyright, contact info, links.

html

```
<footer>
 <p>© 2026 My Site</p>
</footer>
```

**<main>**

Wraps the primary content of the document (excluding headers, nav, sidebars, footers). **Only one**

**<main>** per page.

html

```
<main>
 <article>...</article>
</main>
```

## A typical semantic layout

html

```
<body>
 <header>...</header>
 <nav>...</nav>
 <main>
 <section>
 <article>...</article>
 </section>
 <aside>...</aside>
 </main>
 <footer>...</footer>
</body>
```

## Why semantic HTML matters (know this cold — it's Q2 in Section 5)

1. **Accessibility** — screen readers use these tags to let visually impaired users navigate/jump between page regions.
  2. **SEO** — search engines weight content inside **<article>**/**<main>** differently than generic **<div>**s, improving how your page is indexed.
  3. **Maintainability** — code is self-documenting; another developer (or you, six months later) can understand the page structure at a glance.
-

## 4. Forms

### 4.1 GET vs POST

	GET	POST
Where data goes	Appended to the URL as a query string ( <code>?name=chetan</code> )	Sent in the request body (not visible in URL)
Visibility	Visible in browser history, URL bar, server logs	Hidden from URL
Data limit	Limited (~2000 chars, browser/server dependent)	Effectively no limit
Bookmarkable / shareable	Yes	No
Cacheable	Yes, by default	No, by default
Idempotent	Yes — safe to repeat, doesn't change server state	No — repeating can create duplicate actions (e.g. double form submit)
Typical use	Search forms, filtering, fetching data	Login, signup, file upload, anything sensitive or state-changing

html

```
<!-- GET: data appended as ?query=laptops -->
<form action="/search" method="GET">
 <input type="text" name="query">
</form>

<!-- POST: data sent in the request body -->
<form action="/login" method="POST">
 <input type="password" name="password">
</form>
```

### 4.2 Input Types

`<input>`'s behavior is entirely controlled by `type`:

Type	Purpose
<code>text</code>	Single-line free text
<code>password</code>	Masks characters as dots
<code>email</code>	Validates email format, shows email keyboard on mobile
<code>number</code>	Numeric input, with up/down steppers
<code>tel</code>	Phone number (no built-in validation, but mobile shows number pad)
<code>url</code>	Validates URL format
<code>checkbox</code>	Multiple independent selections
<code>radio</code>	Single selection from a group (grouped by same <code>name</code> )
<code>date</code> / <code>time</code> / <code>month</code> / <code>week</code>	Native date/time pickers
<code>file</code>	File upload
<code>range</code>	Slider between min/max
<code>color</code>	Native color picker
<code>search</code>	Styled like a search box, adds a clear (×) button
<code>hidden</code>	Not shown to user, but submitted with the form (useful for tokens, IDs)
<code>submit</code> / <code>reset</code> / <code>button</code>	Button variants

html

```
<input type="checkbox" name="subscribe" checked>
<input type="radio" name="gender" value="male">
<input type="radio" name="gender" value="female">
```

### 4.3 `required`

A **boolean attribute** that triggers native HTML5 form validation — the form won't submit (and the browser shows a validation message) until the field has a value.

html

```
<input type="text" name="username" required>
```

**Interview note:** This is *client-side* validation only — it can be bypassed (e.g. via dev tools or a direct API call), so **server-side validation is always still required** for real security.

#### 4.4 `placeholder`

Shows faint hint text *inside* the input before the user types. It **disappears once the user starts typing** and is **not a submitted value**.

html

```
<input type="email" placeholder="you@example.com">
```

**Interview trap:** Placeholder is often confused with a label. It is **not an accessible replacement for `<label>`** — once a user starts typing, the hint vanishes, and screen readers don't treat it the same way as an associated label. Never rely on placeholder alone for critical field context.

#### 4.5 `<label>`

Associates descriptive text with a form control — critical for accessibility and usability.

**Method 1 — `for`/`id` pairing:**

html

```
<label for="email">Email:</label>
<input type="email" id="email" name="email">
```

**Method 2 — wrapping:**

html

```
<label>
 Email:
 <input type="email" name="email">
</label>
```

**Why it matters:**

- Clicking the label text **focuses/activates** the associated input (try clicking a checkbox's label — it toggles the box).
- Screen readers announce the label when the input receives focus, so visually impaired users know what the field is for.

- Without labels, a form is genuinely difficult or impossible to use with assistive technology — this is a very common accessibility interview probe.
- 

## 5. Interview Questions & Answers

### Q1. Difference between `<div>` and `<span>`?

<code>&lt;div&gt;</code>	<code>&lt;span&gt;</code>
<b>Block-level</b> — starts on a new line, takes full available width	<b>Inline</b> — stays in the flow of text, only as wide as its content
Used to group larger sections/blocks of content	Used to style/target a small piece of text within a line
Can contain other block or inline elements	Typically wraps just text or small inline elements

**Model answer:** "`<div>` is a block-level container used to group larger chunks of content for layout purposes — it always starts on a new line and takes up the full width available. `<span>` is an inline container used to wrap a small piece of content, like a word or phrase, without disrupting the surrounding text flow — for example, highlighting one word in a sentence with CSS."

---

### Q2. What is semantic HTML?

**Model answer:** "Semantic HTML means using tags that describe the actual meaning and role of the content, rather than generic containers like `<div>` and `<span>`. Tags like `<header>`, `<nav>`, `<article>`, and `<footer>` tell the browser, search engines, and assistive technologies what each part of the page is, not just how it should look. This improves accessibility for screen reader users, helps SEO because search engines can better understand page structure, and makes code more readable and maintainable for other developers."

---

### Q3. Why use `alt` in images?

**Model answer:** "The `alt` attribute serves three purposes: **accessibility** — screen readers read the `alt` text aloud so visually impaired users know what the image shows; **fallback** — if the image fails to load (broken link, slow connection), the browser displays the `alt` text instead; and **SEO** — search engines can't 'see' images, so they rely on `alt` text to understand and index image content. For purely decorative images that add no information, best practice is to use an empty `alt=""` so screen readers skip over them instead of reading a meaningless description."

---

#### Q4. Difference between `id` and `class`?

`id`

`class`

Must be **unique** — one element per page

**Reusable** — many elements can share a class

One `id` per element

One element can have **multiple classes** (space-separated)

Used for: unique JS targeting (`getElementById`), CSS for one specific element, in-page anchor links (`#section1`)

Used for: styling/behavior shared across many elements (`getElementsByClassName`, `querySelectorAll`)

Higher CSS **specificity** than class

Lower specificity than id

html

```
<div id="main-header" class="card highlighted">...</div>
```

**Model answer:** "`id` uniquely identifies a single element on the page — it should only be used once — while `class` is meant to be reused across multiple elements that share styling or behavior. An element can have only one `id` but multiple classes. In CSS, `id` selectors also have higher specificity than class selectors, which matters when there's a styling conflict."

#### Q5. What are meta tags?

**Model answer:** "Meta tags live inside `<head>` and provide metadata about the document — information that isn't rendered on the page itself but is used by browsers, search engines, and social platforms."

Common examples:

html

```
<meta charset="UTF-8">
<!-- Sets character encoding so special characters render correctly -->

<meta name="viewport" content="width=device-width, initial-scale=1.0">
<!-- Makes the page responsive on mobile devices -->

<meta name="description" content="A short summary of the page">
<!-- Shown in search engine result snippets -->

<meta name="keywords" content="html, css, javascript">
<!-- Historically used for SEO, largely ignored by modern search engines now -->

<meta name="author" content="Chetan">
<!-- Identifies the page author -->

<meta property="og:title" content="Page Title">
<meta property="og:image" content="preview.jpg">
<!-- Open Graph tags – control how the page looks when shared on social media (Facebook, Twitter, LinkedIn, etc.) -->
```

## 6. Quick-Fire Revision Table

Use this as a last-minute scan before walking into the interview.

Concept	One-line answer
HTML	Markup language that structures web content — not a programming language
<code>&lt;!DOCTYPE html&gt;</code>	Forces standards mode rendering
Element vs Tag	Tag = the bracket keyword; Element = tag + content + closing tag
<code>&lt;div&gt;</code> vs <code>&lt;span&gt;</code>	Block vs inline, no semantic meaning either way
Semantic HTML	Tags that convey meaning ( <code>&lt;article&gt;</code> , <code>&lt;nav&gt;</code> ) → better accessibility + SEO
<code>alt</code> attribute	Accessibility + broken-image fallback + SEO
<code>id</code> vs <code>class</code>	Unique/one-per-page vs reusable/many-per-page; id has higher CSS specificity
GET vs POST	GET = URL, visible, idempotent; POST = body, hidden, for sensitive/state-changing actions
<code>placeholder</code> vs <code>label</code>	Placeholder disappears on typing and isn't accessible-safe; label is persistent and screen-reader friendly
<code>required</code>	Client-side only — never trust it alone, validate on the server too
<code>&lt;main&gt;</code>	Only one per page — the primary content region
Meta viewport tag	Required for responsive/mobile-friendly rendering

**Prep tip:** In interviews, structure your answers as: *definition* → *short example* → *why it matters* (*accessibility/SEO/UX/security*). That third part is what separates a memorized answer from one that shows real understanding.